

Part A: Produce the Insight

1. Prepare the data
2. Make one justified cleaning decision
3. Ask an analytical question
4. Analyse before you plot
5. Choose the visual deliberately
6. Challenge the visual
7. Communicate the result

```
In [6]: from google.colab import files
csv = files.upload()

[Choose File] No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Happiness.csv to Happiness.csv

1. Load the dataset into Pandas and preview
```

```
In [7]: import pandas as pd
bs = pd.read_csv('Happiness.csv')
bs

Out[7]:
```

	Country	Region	Happiness Rank	Happiness Score	Lower Confidence Interval	Upper Confidence Interval	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
0	Denmark	Western Europe	1	7.528	7.460	7.592	1.44178	1.16374	0.79504	0.57941	0.44453	0.36171	2.73939
1	Switzerland	Western Europe	2	7.509	7.428	7.590	1.52733	1.14524	0.86303	0.58557	0.41203	0.28083	2.69463
2	Iceland	Western Europe	3	7.501	7.333	7.669	1.42666	1.18326	0.86733	0.56624	0.14975	0.47678	2.83137
3	Norway	Western Europe	4	7.498	7.421	7.575	1.57744	1.12690	0.79579	0.59609	0.35776	0.37895	2.66465
4	Finland	Western Europe	5	7.413	7.351	7.475	1.40598	1.13464	0.81091	0.57104	0.41004	0.25492	2.82596
...	...	...	...	...	...	...	...	...	...	...	...	...	...
152	Benin	Sub-Saharan Africa	153	3.454	3.404	3.564	0.39499	0.10419	0.21028	0.39747	0.06681	0.20180	2.10812
153	Afghanistan	Southern Asia	154	3.360	3.288	3.432	0.38227	0.11037	0.17344	0.16430	0.07112	0.31268	2.14558
154	Togo	Sub-Saharan Africa	155	3.303	3.192	3.414	0.28123	0.00000	0.24811	0.34678	0.11587	0.17517	2.13540
155	Syria	Middle East and Northern Africa	156	3.069	2.936	3.202	0.74719	0.14866	0.62994	0.06912	0.17233	0.46397	0.81789
156	Burundi	Sub-Saharan Africa	157	2.905	2.732	3.078	0.06831	0.23442	0.15747	0.04320	0.09419	0.20290	2.10404

157 rows × 13 columns

1. Check its shape, column names, data types and missing values.
- bs.null().sum() - Used to check if the dataset has any null values
- bs.dtypes - Used to check the datatype of each column

```
In [8]: print(bs.shape, bs.columns, bs.dtypes, bs.isnull().sum(), sep='\n\n')
(157, 13)

Index(['Country', 'Region', 'Happiness Rank', 'Happiness Score',
      'Lower Confidence Interval', 'Upper Confidence Interval',
      'Economy (GDP per Capital)', 'Family', 'Health (Life Expectancy)',
      'Freedom', 'Trust (Government Corruption)', 'Generosity',
      'Dystopia Residual'],
      dtype='object')

Country      object
Region       object
Happiness Rank      int64
Happiness Score    float64
Lower Confidence Interval    float64
Upper Confidence Interval    float64
Economy (GDP per Capital)    float64
Family         float64
Health (Life Expectancy)     float64
Freedom        float64
Trust (Government Corruption) float64
Generosity      float64
Dystopia Residual    float64
dtype: object

Country      0
Region       0
Happiness Rank      0
Happiness Score    0
Lower Confidence Interval    0
Upper Confidence Interval    0
Economy (GDP per Capital)    0
Family         0
Health (Life Expectancy)     0
Freedom        0
Trust (Government Corruption) 0
Generosity      0
Dystopia Residual    0
dtype: int64

2. Identify one potential data-quality issue or consistency check. Apply an appropriate cleaning step only if one is needed. Use a comment to explain why you made the change. If no change is needed, explain how you established that.
```

After checking for duplicates there were a couple of columns that had a small amount of duplicate values, most of which could be ignored as it's completely possible for multiple countries to have the same values in each category.

However, I thought it would be best to check the three duplicates that appear in the Happiness Rank column to confirm if these duplicates are mistakes or because they share the same rank due to the score being the same. The answer is the latter. This is also re-confirmed as there is no rank 35, 58, or 146.

As there are no duplicate errors, NaN values, incorrect datatypes, I have come to the conclusion the dataset is clean and there is no need for any cleaning steps to take.

```
In [9]: bs.apply(pd.Series.duplicated).sum()
0

Out[9]:
```

	Country	Region	Happiness Rank	Happiness Score	Lower Confidence Interval	Upper Confidence Interval	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
			0										
	Country	0											
	Region	147											
	Happiness Rank	3											
	Happiness Score	3											
	Lower Confidence Interval	3											
	Upper Confidence Interval	3											
	Economy (GDP per Capita)	0											
	Family	0											
	Health (Life Expectancy)	1											
	Freedom	0											
	Trust (Government Corruption)	1											
	Generosity	0											
	Dystopia Residual	0											

dtype: int64

```
In [10]: bs[bs['Happiness Rank'].duplicated(keep=False)]

Out[10]:
```

	Country	Region	Happiness Rank	Happiness Score	Lower Confidence Interval	Upper Confidence Interval	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	Trust (Government Corruption)	Generosity	Dystopia Residual
33	Saudi Arabia	Middle East and Northern Africa	34	6.379	6.287	6.471	1.49863	0.84329	0.59267	0.37904	0.30008	0.15457	2.61482
34	Taiwan	Eastern Asia	34	6.379	6.305	6.453	1.39729	0.92624	0.79565	0.32377	0.08630	0.25495	2.61523
56	Poland	Central and Eastern Europe	57	5.835	5.749	5.921	1.24585	1.04685	0.69058	0.45190	0.05500	0.14443	2.20035
57	South Korea	Eastern Asia	57	5.835	5.747	5.923	1.35948	0.72194	0.88645	0.25168	0.07716	0.18824	2.35015
144	Burkina Faso	Sub-Saharan Africa	145	3.739	3.647	3.831	0.31985	0.63054	0.21297	0.33370	0.12533	0.24353	1.87319
145	Uganda	Sub-Saharan Africa	145	3.739	3.629	3.849	0.34719	0.90081	0.19625	0.43653	0.06442	0.27102	1.51416

```
In [11]: print(bs['Happiness Rank'].iloc[33:36], bs['Happiness Rank'].iloc[56:59], bs['Happiness Rank'].iloc[144:147], sep='\n\n')
33 34
34 34
35 36
Name: Happiness Rank, dtype: int64

56 57
57 57
58 59
Name: Happiness Rank, dtype: int64

144 145
145 145
146 147
Name: Happiness Rank, dtype: int64

3. Ask an analytical question

Write one question that the dataset can answer. Examples include: How are marks distributed? Which region has the most records? How do average values compare between categories? Your question should determine the analysis and chart you choose.
```

Question: Which region has the highest average Happiness Score?

4. Analyse before you plot

Use Pandas to calculate the information needed to answer your question. This might involve filtering, value_counts(), groupby(), mean(), median(), min(), max(), describe() or another method covered in the workshop.

```
In [12]: avg_hs = bs.groupby(['Region'])['Happiness Score'].mean().round(2).sort_values(ascending=False).reset_index()
avg_hs

Out[12]:
```

	Region	Happiness Score
0	Australia and New Zealand	7.32
1	North America	7.25
2	Western Europe	6.69
3	Latin America and Caribbean	6.10
4	Eastern Asia	5.62
5	Middle East and Northern Africa	5.39
6	Central and Eastern Europe	5.37
7	Southeastern Asia	5.34
8	Southern Asia	4.56
9	Sub-Saharan Africa	4.14

5. Choose the visual deliberately

Create an appropriate chart using Matplotlib, Seaborn or Pandas plotting. Give it a useful title and label the axes where appropriate. Under the chart, write 2-3 sentences explaining why the chart suits your question and what it shows.

```
In [13]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15, 5))
sns.barplot(y='Region', x='Happiness Score', data = avg_hs)
plt.title('Average Happiness Score by Region')
plt.show()

Out[13]:
```

6. Challenge the visual

Create a second, different chart from the same dataset. Decide whether the second chart communicates the information better or worse than the first and explain why. The aim is to practise judgement, not simply make more charts.

Although you could reach the same conclusion using the line chart below, it presents the information in a less clear and potentially more confusing way. At a quick glance, the connected line may suggest that the chart is showing how happiness scores change over time, rather than comparing the average happiness score across different regions.

```
In [14]: plt.figure(figsize=(15, 5))
plt.plot(avg_hs['Region'], avg_hs['Happiness Score'], marker='o')
plt.title('Average Happiness Score by Region')
plt.xticks(rotation=45)
plt.show()

Out[14]:
```

7. Communicate the result

Write a short three-point update to your manager. What did you investigate? What did you find? What would you recommend checking or analysing next?

```
Initial Investigation

After reviewing the available data, I wanted to investigate the happiness score across each region to identify which regions were the happiest and which had the lowest levels of happiness.

Results

After filtering the data and grouping it by region, the results showed that Australia & New Zealand had the highest average happiness score at 7.32, while Sub-Saharan Africa had the lowest at 4.14.

Further Analysis

We can dive deeper into the data by using the happiness score as a reference point to investigate whether any of the other statistics are contributing to the differences in happiness between regions. For example, we could investigate whether there is a correlation between happiness and the average GDP per region, or between happiness and average trust in government.

Part B: Interview and Retrieval Questions

8. Day 1: Data types Interview question: Why should you check data types before creating a chart or calculation? Give a workplace example.

You should check data types before creating a chart or calculation because the incorrect data type can cause incorrect results or errors. For example, if a column containing sales figures is stored as a string instead of a float or integer, you may not be able to calculate the total sales or create an accurate chart.

9. Day 2: Comparisons

A manager asks for a chart showing only records above a target. Explain how a comparison operator from Day 2 becomes part of the Pandas filtering step.

When you are tasked with filtering data that is above a specific target, comparison operators are one of your best tools as these operators allow you to perform checks that return a true or false and allows you to then determine what happens to the data depending on the outcome of the check.

In this case, the '>' operator, meaning greater than, will be used to perform a check on the value to see if its greater than the target.

10. Day 2: Collections

Where can you see earlier Python collections being used in Pandas code, for example when selecting multiple columns? Give one example.

A python list, created using square brackets containing its values separated by a comma allows pandas to loop through the list selecting each value. If these values were column names, pandas will loop through the names and return each column matching with the matching name in the list.
```

```
11. Day 3: Data quality

Interview question: Why would you use head(), info(), shape or isnull().sum() before visualising data? Explain one risk of skipping those checks.

These functions are very useful for checking the data prior to performing any cleaning or analysis as they provide information regarding the DataFrame.

head() will show the first 5 rows allowing you to quickly check that the data has loaded correctly, where as info(), shape, isnull().sum() provide structural information that allows you to see the size, datatypes, count of null values which leads you to find any errors or missing values quickly.

12. Day 3: Analysis

Why is creating a chart not normally the first step? What analysis or preparation might need to happen before a bar chart of average results by category?

You would create a chart first as you typically need to filter and group the data based on the specific categories before you start to plot the data into charts.
```

13. Day 4: Judgement

A colleague says: "The code ran, so the chart must be correct." How would you respond? Include at least two things an analyst should check beyond whether the code executes.

- The code running doesn't necessarily mean the chart is correct. Its always good practice to double check the chart represents the data accurately. You should ask you self these questions,
- Is the chart showing the data im expecting?
- Is the correct chart type to represent this data, are the label, axes, and scales misleading in any way?

14. Interview question: Using AI

You use AI to generate a Seaborn chart. What must you still be able to check and explain before using that chart in a workplace report?

You should be able to understand and check that the chart is accurate to the data and not misrepresenting the data while still being able to explain what the chart represents.

Stretch Challenge

Write a second business question using a different field or relationship in the dataset. Carry out the analysis, choose a different chart type and explain the insight. Avoid choosing a chart only because you remember its syntax.

The below attempts to determine if there is any correlation between the Trust (Government Corruption) and Happiness Score.

The scatter plot shows a generally positive relationship between happiness score and trust in government. As the happiness score increases, trust also tends to increase. However, there is still variation between countries, particularly among those with lower and middle happiness scores suggesting that while trust may be associated with happiness, it is unlikely to be the only factor influencing the happiness score.

```
In [46]: t_hs = bs[['Trust (Government Corruption)', 'Happiness Score']]
t_hs

Out[46]:
```

	Trust (Government Corruption)	Happiness Score
0	0.44453	7.528
1	0.41203	7.509
2	0.14975	7.501
3	0.35776	7.498
4	0.41004	7.413
...	...	...
152	0.06681	3.484
153	0.07112	3.360
154	0.11587	3.303
155	0.17233	3.069
156	0.09419	2.905

157 rows × 2 columns

```
In [59]: t_hs.plot.scatter(
x='Happiness Score',
y='Trust (Government Corruption)',
s=75,
alpha=0.8,
figsize=(10,6)
)
plt.show()
```